

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO5	Assessment:	Tool 2– INDUSTRY PROBLEM SOLVING
Weightage:	35%	Total Marks:	
CO5 Statement: Create meaningful visualizations using Pandas and Matplotlib, including charts, distributions, relationships, and interactive visual representations for effective communication			
Student Name:	Sania Herbert	Reg. No.:	192324062
Section Batch: /	2023/AI&DS	Date of Submission:	22.08.26
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I Sania Herbert (192324062) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sania

1. Manufacturing Production Analysis

A manufacturing company records daily production, machine operating hours, temperature, energy consumption, and defect count. **Import and explore the dataset using Pandas. Develop visualizations to analyze production trends over time, relationships between machine parameters and defects, and distributions of production values. Use a scatter matrix, lag plot, and autocorrelation plot to identify hidden production patterns and provide recommendations for improving efficiency.**

Objective

Analyze daily manufacturing data containing:

- Date
- Production quantity
- Machine operating hours
- Temperature
- Energy consumption
- Defect count

DATASET DESCRIPTION

Attribute	Description
Date	Date of production
Production	Quantity produced per day
Machine_Hours	Number of hours machines operated
Temperature	Operating/environmental temperature
Energy_Consumption	Energy consumed during production
Defect_Count	Number of defective products

Production Trend Analysis

A **line chart** is appropriate for time-series data because it shows how a variable changes over time.

The production trend can help identify:

- Increasing production
- Decreasing production
- Production fluctuations
- Seasonal or periodic behavior
- Unusual production days

Distribution Analysis

A **histogram** represents the frequency distribution of numerical data.

For production data, it shows how frequently different production levels occur.

A histogram helps identify:

- Most common production levels
- Data spread
- Skewness
- Extreme values
- Possible unusual observations

Scatter Plot Analysis

A scatter plot represents the relationship between two numerical variables.

For example:

Machine Hours vs Defect Count

This can help determine whether longer machine operation is associated with more defects.

Another useful relationship is:

Temperature vs Defect Count

LAG PLOT

A lag plot compares a data value with its previous observation.

For production:

Today's Production vs Previous Day's Production

It is particularly useful for time-series data.

Interpretation

If the points show a clear pattern, current production may depend on previous production.

If the points appear randomly scattered, there may be little relationship between consecutive production values.

Autocorrelation Plot

Autocorrelation measures the relationship between observations of a time series at different time lags.

For example, it can determine whether production on one day is related to production several days earlier.

Importance

Autocorrelation can reveal:

- Repeating production patterns
- Time dependency
- Periodicity
- Persistent fluctuations
- Possible seasonal behavior

A high autocorrelation at a particular lag indicates that observations separated by that lag may have a meaningful relationship.

PYTHON CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pandas.plotting import scatter_matrix, lag_plot, autocorrelation_plot

# =====
# PART 1: GENERATE AND SAVE THE DATASET
# =====
print("Generating dataset...")
np.random.seed(42)
days = 365
dates = pd.date_range(start='2025-01-01', periods=days, freq='D')

# Create baseline patterns for the manufacturing variables
time_pattern = np.sin(np.linspace(0, 2 * np.pi * (days/7), days))
daily_production = 1000 + (time_pattern * 200) + np.random.normal(0, 50, days)
machine_hours = daily_production / 50 + np.random.normal(0, 1.5, days)
temperature = 22 + np.sin(np.linspace(0, np.pi, days)) * 10 + np.random.normal(0, 3, days)
```

```
energy_consumption = (machine_hours * 15) + (temperature * 5) +  
np.random.normal(0, 20, days)  
defect_count = np.maximum(0, (temperature - 25) * 2 + (machine_hours - 18)  
* 1.5 + np.random.normal(0, 2, days)).astype(int)
```

```
# Compile into a Pandas DataFrame  
df = pd.DataFrame({  
    'Date': dates,  
    'Production_Units': daily_production,  
    'Machine_Hours': machine_hours,  
    'Temperature_C': temperature,  
    'Energy_kWh': energy_consumption,  
    'Defect_Count': defect_count  
})
```

```
# Save the dataset to a CSV file  
file_name = 'manufacturing_production_data.csv'  
df.to_csv(file_name, index=False)  
print(f"Dataset successfully saved as '{file_name}'!\n")
```

```
# =====  
# PART 2: IMPORT AND EXPLORE THE DATASET  
# =====  
print("Importing and analyzing dataset...")
```

```
# Import the dataset we just created, setting Date as the index  
data = pd.read_csv(file_name, parse_dates=['Date'], index_col='Date')
```

```
# Print basic exploration metrics  
print("\n--- Dataset Info ---")  
print(data.info())  
print("\n--- Statistical Summary ---")  
print(data.describe().round(2))
```

```
# Set plotting style  
plt.style.use('seaborn-v0_8-darkgrid')
```

```
# 1. Line Plot: Production Trends Over Time  
plt.figure(figsize=(14, 5))  
plt.plot(data.index, data['Production_Units'], color='#1f77b4', linewidth=1.5)  
plt.title('Daily Production Trends Over 1 Year', fontsize=14, fontweight='bold')  
plt.xlabel('Date')  
plt.ylabel('Units Produced')  
plt.tight_layout()  
plt.show()
```

```
# 2. Scatter Matrix: Relationships & Distributions  
# Analyzes the correlation between machine parameters and defects
```

```

scatter_matrix(data, alpha=0.6, figsize=(12, 12), diagonal='kde',
color='#2ca02c')
plt.suptitle('Scatter Matrix: Relationships Between Manufacturing Parameters',
y=1.02, fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

```

3. Lag Plot: Hidden Production Patterns

```

# Checks if production on a given day is correlated with the previous day
plt.figure(figsize=(6, 6))
lag_plot(data['Production_Units'], c='#d62728', alpha=0.7)
plt.title('Lag Plot: Production (Day t) vs Production (Day t+1)', fontsize=12,
fontweight='bold')
plt.tight_layout()
plt.show()

```

4. Autocorrelation Plot: Weekly Seasonality

```

# Identifies repeating cyclical patterns in production volume
plt.figure(figsize=(10, 5))
autocorrelation_plot(data['Production_Units'], color='#9467bd')
plt.title('Autocorrelation Plot: Identifying Production Cycles', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.show()

```

OUTPUT:

Generating dataset...

Dataset successfully saved as 'manufacturing_production_data.csv'!

Importing and analyzing dataset...

--- Dataset Info ---

<class 'pandas.DataFrame'>

DatetimeIndex: 365 entries, 2025-01-01 to 2025-12-31

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	Production_Units	365 non-null	float64
1	Machine_Hours	365 non-null	float64
2	Temperature_C	365 non-null	float64
3	Energy_kWh	365 non-null	float64
4	Defect_Count	365 non-null	int64

dtypes: float64(4), int64(1)

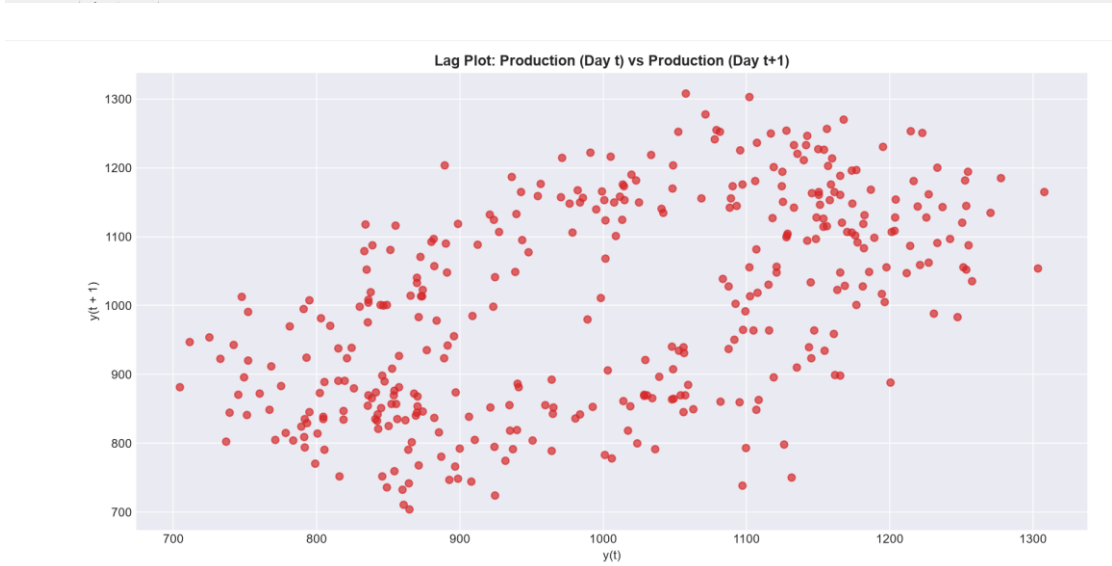
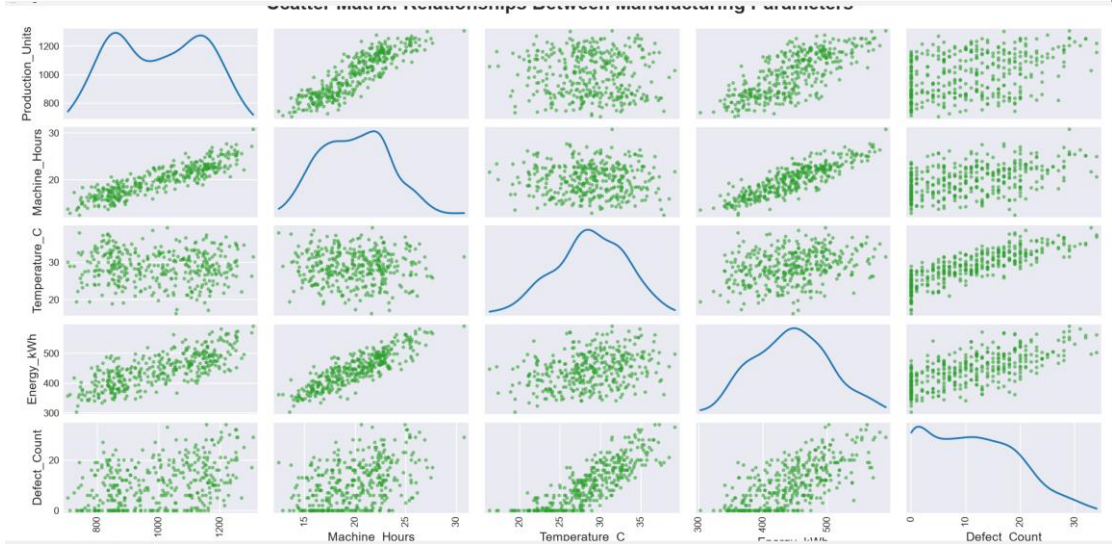
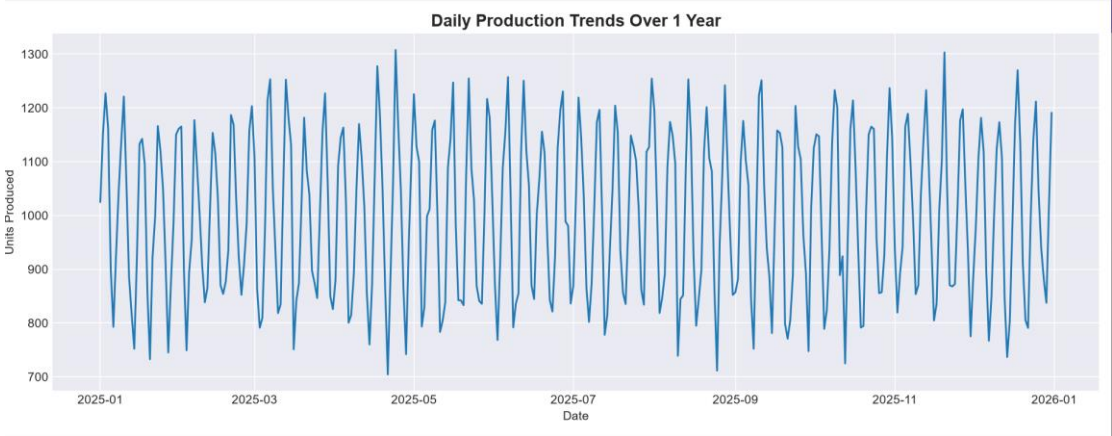
memory usage: 17.1 KB

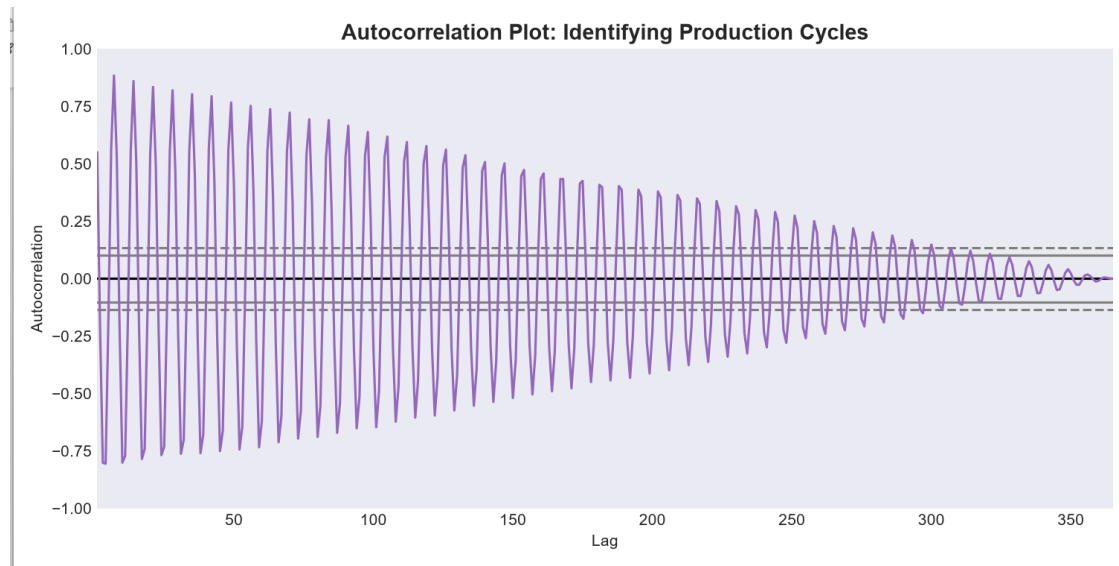
None

--- Statistical Summary ---

	Production_Units	Machine_Hours	Temperature_C	Energy_kWh	Defect_Count
--	------------------	---------------	---------------	------------	--------------

count	365.00	365.00	365.00	365.00	365.00
mean	1000.93	19.96	28.73	444.70	10.95
std	149.19	3.26	4.33	56.32	8.62
min	704.28	12.43	16.26	302.75	0.00
25%	865.59	17.43	26.15	403.28	3.00
50%	1005.77	20.00	28.90	445.49	10.00
75%	1133.05	22.42	32.04	483.63	17.00
max	1307.80	30.77	39.20	590.69	34.00





2. E-Commerce Sales Pattern Analysis

An e-commerce company has historical data containing **date**, **product category**, **quantity sold**, **revenue**, **discount**, and **customer rating**. Analyze the evolution of sales over time and identify relationships between discount, quantity, and revenue. Use appropriate **bar charts**, **line charts**, **distribution plots**, **frequency plots**, and **scatter matrices** to identify important business

• Objectives

- To import and explore e-commerce sales data.
- To analyze sales evolution over time.
- To compare revenue across product categories.
- To study the distribution of quantity sold.
- To investigate the relationship between discount and quantity sold.
- To investigate the relationship between discount and revenue.
- To analyze customer rating patterns.
- To use scatter matrix visualization to identify relationships.
- To identify important business patterns.
- To provide recommendations for improving sales performance.

3. Dataset Description

Attribute	Description
Date	Date of transaction/sale

Attribute	Description
Product_Category	Category of the product
Quantity_Sold	Number of products sold
Revenue	Revenue generated
Discount	Discount offered
Customer_Rating	Customer rating

Sales Trend Analysis

A **line chart** is used to visualize revenue or quantity sold over time.

It helps identify:

- Sales growth
- Sales decline
- Periodic fluctuations
- Peak sales periods
- Low-sales periods

Category-Wise Revenue Analysis

A bar chart **is useful for comparing different product categories.**

Quantity Distribution

A histogram can be used to analyze how frequently different quantities are sold.

It helps identify:

- Typical order quantity
- High-volume purchases
- Low-volume purchases
- Distribution shape
- Unusual values

Discount vs Quantity Sold

A scatter plot can be used to study whether increasing discounts result in higher quantities sold.

Possible interpretations

Positive relationship:

Higher discounts → Higher quantity sold

This suggests that discounts may encourage customers to purchase more.

Weak relationship:

Discounts have limited influence on quantity sold.

Negative relationship:

Higher discounts are associated with lower quantities, possibly because discounts are being applied to less popular products.

Discount vs Revenue

Discount does not necessarily increase revenue.

For example, a high discount may increase quantity sold but reduce revenue per item.

Therefore, the company should examine both:

Discount → Quantity Sold

and

Discount → Revenue

rather than considering quantity alone.

Business importance

This helps determine whether promotional campaigns are actually profitable.

9. Customer Rating Analysis

Customer ratings can be analyzed using a histogram or distribution plot.

Ratings can provide information about customer satisfaction.

For example:

- High average rating → Strong customer satisfaction
- Low average rating → Possible product/service problems
- Large variation → Inconsistent customer experience

Customer rating can also be compared with revenue or quantity sold to investigate whether highly rated products generate more sales.

10. Scatter Matrix

A scatter matrix can simultaneously compare:

- Quantity Sold
- Revenue

- Discount
- Customer Rating

This allows the analyst to identify relationships that may not be obvious from individual charts.

Python code:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from pandas.plotting import scatter_matrix
```

```
# =====
```

```
# 1. LOAD DATASET
```

```
# =====
```

```
df = pd.read_csv(
```

```
    "C:\\Users\\DELL\\Documents\\dataset for question 2.txt",
```

```
    sep=None,
```

```
    engine="python"
```

```
)
```

```
# Remove spaces from column names
```

```
df.columns = df.columns.str.strip()
```

```
print("=" * 60)
```

```
print("E-COMMERCE SALES PATTERN ANALYSIS")
```

```
print("=" * 60)
```

```
# =====
```

``` # 2. DATA EXPLORATION ```

```
# =====
```

```
print("\nFirst 5 Records:")
```

```
print(df.head())
```

```
print("\nDataset Shape:")
```

```
print(df.shape)
```

```
print("\nColumn Names:")
```

```
print(df.columns.tolist())
```

```
print("\nMissing Values:")
```

```
print(df.isnull().sum())
```

```
# =====
```

``` # 3. DATE CONVERSION ```

```
# =====
```

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
df = df.sort_values("Date")
```

```
print("\nDate Conversion Successful!")
```

```
# =====
```

```
# 4. STATISTICAL SUMMARY
```

```
# =====
```

```
print("\nStatistical Summary:")
```

```
print(df.describe())
```

```
daily_sales = df.groupby("Date")["Revenue"].sum()
```

```
plt.figure(figsize=(12, 5))
```

```
plt.plot(
```

```
    daily_sales.index,
```

```
    daily_sales.values,
```

```
    marker="o"
```

```
)
```

```
plt.title("E-Commerce Revenue Trend Over Time")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Revenue")
```

```
plt.xticks(rotation=45)
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
category_revenue = (
```

```
df.groupby("Product_Category")["Revenue"]  
  
.sum()  
  
.sort_values(ascending=False)  
  
)
```

```
print("\nRevenue by Product Category:")
```

```
print(category_revenue)
```

```
plt.figure(figsize=(9, 5))
```

```
plt.bar(  
  
    category_revenue.index,  
  
    category_revenue.values  
  
)
```

```
plt.title("Revenue by Product Category")
```

```
plt.xlabel("Product Category")
```

```
plt.ylabel("Total Revenue")
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
category_quantity = (
```

```
    df.groupby("Product_Category")["Quantity_Sold"]
```

```
    .sum()
```

```
    .sort_values(ascending=False)
```

)

```
plt.figure(figsize=(9, 5))
```

```
plt.bar(  
    category_quantity.index,  
    category_quantity.values  
)
```

```
plt.title("Quantity Sold by Product Category")
```

```
plt.xlabel("Product Category")
```

```
plt.ylabel("Total Quantity Sold")
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
plt.figure(figsize=(9, 5))
```

```
plt.hist(  
    df["Revenue"],  
    bins=10  
)
```

```
plt.title("Distribution of Revenue")
```

```
plt.xlabel("Revenue")
```

```
plt.ylabel("Frequency")
```

```
plt.grid(axis="y")  
plt.tight_layout()  
plt.show()  
plt.figure(figsize=(9, 5))
```

```
plt.hist(  
    df["Quantity_Sold"],  
    bins=10  
)
```

```
plt.title("Distribution of Quantity Sold")  
plt.xlabel("Quantity Sold")  
plt.ylabel("Frequency")  
plt.grid(axis="y")  
plt.tight_layout()  
plt.show()  
plt.figure(figsize=(9, 5))
```

```
plt.hist(  
    df["Discount"],  
    bins=10  
)
```

```
plt.title("Distribution of Discounts")  
plt.xlabel("Discount (%)")
```



```
plt.ylabel("Frequency")
```

```
plt.grid(axis="y")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
plt.figure(figsize=(9, 5))
```

```
plt.hist(
```

```
    df["Customer_Rating"],
```

```
    bins=10
```

```
)
```

```
plt.title("Distribution of Customer Ratings")
```

```
plt.xlabel("Customer Rating")
```

```
plt.ylabel("Frequency")
```

```
plt.grid(axis="y")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
plt.figure(figsize=(8, 5))
```

```
plt.scatter(
```

```
    df["Discount"],
```

```
    df["Quantity_Sold"]
```

```
)
```

```
plt.title("Discount vs Quantity Sold")
```

```
plt.xlabel("Discount (%)")  
plt.ylabel("Quantity Sold")  
plt.grid(True)  
plt.tight_layout()  
plt.show()  
plt.figure(figsize=(8, 5))
```

```
plt.scatter(  
    df["Discount"],  
    df["Revenue"]  
)
```

```
plt.title("Discount vs Revenue")  
plt.xlabel("Discount (%)")  
plt.ylabel("Revenue")  
plt.grid(True)  
plt.tight_layout()  
plt.show()  
plt.figure(figsize=(8, 5))
```

```
plt.scatter(  
    df["Quantity_Sold"],  
    df["Revenue"]  
)
```

```
plt.title("Quantity Sold vs Revenue")
```

```
plt.xlabel("Quantity Sold")
```

```
plt.ylabel("Revenue")
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
numeric_columns = [
```

```
    "Quantity_Sold",
```

```
    "Revenue",
```

```
    "Discount",
```

```
    "Customer_Rating"
```

```
]
```

```
scatter_matrix(
```

```
    df[numeric_columns],
```

```
    figsize=(12, 10),
```

```
    diagonal="hist"
```

```
)
```

```
plt.suptitle(
```

```
    "Scatter Matrix of E-Commerce Variables",
```

```
    y=1.02
```

```
)
```

```
plt.tight_layout()
```

```

plt.show()

print("\n" + "=" * 60)

print("FREQUENCY ANALYSIS")

print("=" * 60)


print("\nProduct Category Frequency:")

print(df["Product_Category"].value_counts())


print("\nQuantity Sold Frequency:")

print(df["Quantity_Sold"].value_counts().sort_index())


print("\nCustomer Rating Frequency:")

print(df["Customer_Rating"].value_counts().sort_index())

total_revenue = df["Revenue"].sum()

total_quantity = df["Quantity_Sold"].sum()

average_discount = df["Discount"].mean()

average_rating = df["Customer_Rating"].mean()


highest_revenue_category = category_revenue.idxmax()


print("\n" + "=" * 60)

print("FINAL BUSINESS ANALYSIS")

print("=" * 60)


print(f"Total Revenue      : {total_revenue:.2f}")

```

```
print(f"Total Quantity Sold : {total_quantity}")

print(f"Average Discount   : {average_discount:.2f}%")

print(f"Average Rating     : {average_rating:.2f}")

print(f"Top Revenue Category: {highest_revenue_category}")
```

=====

E-COMMERCE SALES PATTERN ANALYSIS

=====

First 5 Records:

	Date	Product_Category	Quantity_Sold	Revenue	Discount	Customer_Rating
0	2026-02-01	Electronics	8	1248.5	10	4.5
1	2026-02-02	Clothing	12	594.0	15	4.2
2	2026-02-03	Home & Kitchen	6	459.0	8	4.6
3	2026-02-04	Beauty	15	510.0	12	4.1
4	2026-02-05	Sports	9	526.5	10	4.4

Dataset Shape:

(30, 6)

Column Names:

['Date', 'Product_Category', 'Quantity_Sold', 'Revenue', 'Discount', 'Customer_Rating']

Missing Values:

Date 0

Product_Category 0

Quantity_Sold 0

Revenue 0

Discount 0

Customer_Rating 0

dtype: int64

Date Conversion Successful!

Statistical Summary:

	Date	Quantity_Sold	Revenue	Discount	Customer_Rating
count	30	30.000000	30.000000	30.000000	30.000000
mean	2026-02-15 12:00:00	11.766667	770.625000	12.466667	4.410000
min	2026-02-01 00:00:00	5.000000	374.000000	8.000000	4.000000
25%	2026-02-08 06:00:00	9.000000	552.500000	10.000000	4.200000
50%	2026-02-15 12:00:00	11.500000	694.875000	12.000000	4.450000
75%	2026-02-22 18:00:00	14.750000	841.500000	15.000000	4.600000
max	2026-03-02 00:00:00	20.000000	1944.000000	20.000000	4.900000
std	NaN	4.116270	330.779648	3.266690	0.250998

Revenue by Product Category:

Product_Category

Electronics 7566.50

Sports 4329.00

Clothing 4193.75

Home & Kitchen 3901.50

Beauty 3128.00

Name: Revenue, dtype: float64

=====

FREQUENCY ANALYSIS

=====

Product Category Frequency:

Product_Category

Electronics 6

Clothing 6

Home & Kitchen 6

Beauty 6

Sports 6

Name: count, dtype: int64

Quantity Sold Frequency:

Quantity_Sold

5 1

6 2

7 2

8 2

9 3

10 3

11 2

12 3

13 2

14 2

15 2

16 2

17 1

18 1

20 2

Name: count, dtype: int64

Customer Rating Frequency:

Customer_Rating

4.0 3

4.1 3

4.2 3

4.3 3

4.4 3

4.5 5

4.6 5

4.7 3

4.8 1

4.9 1

Name: count, dtype: int64

=====

FINAL BUSINESS ANALYSIS

=====

Total Revenue : 23118.75

Total Quantity Sold : 353

Average Discount : 12.47%

Average Rating : 4.41

Top Revenue Category: Electronics

